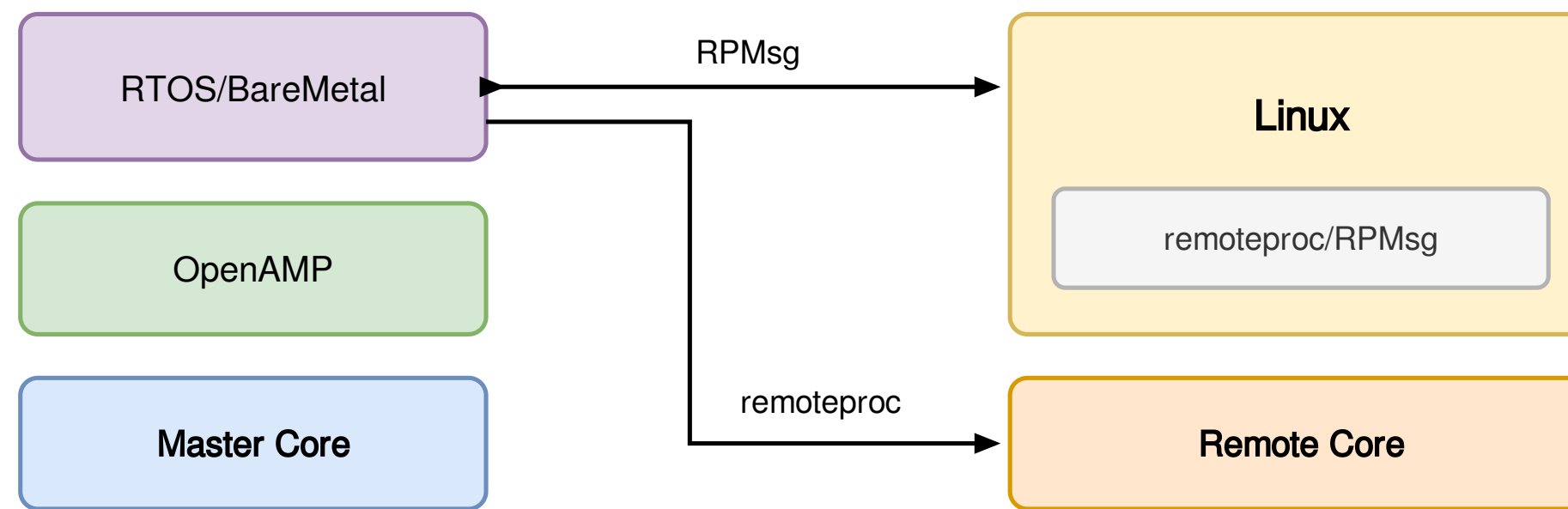


Investigating Real-Time Communication Between Asymmetric Processors with OpenAMP

***Project A2
Luigi Auggiero
M63001866***

Canonical OpenAMP Architecture

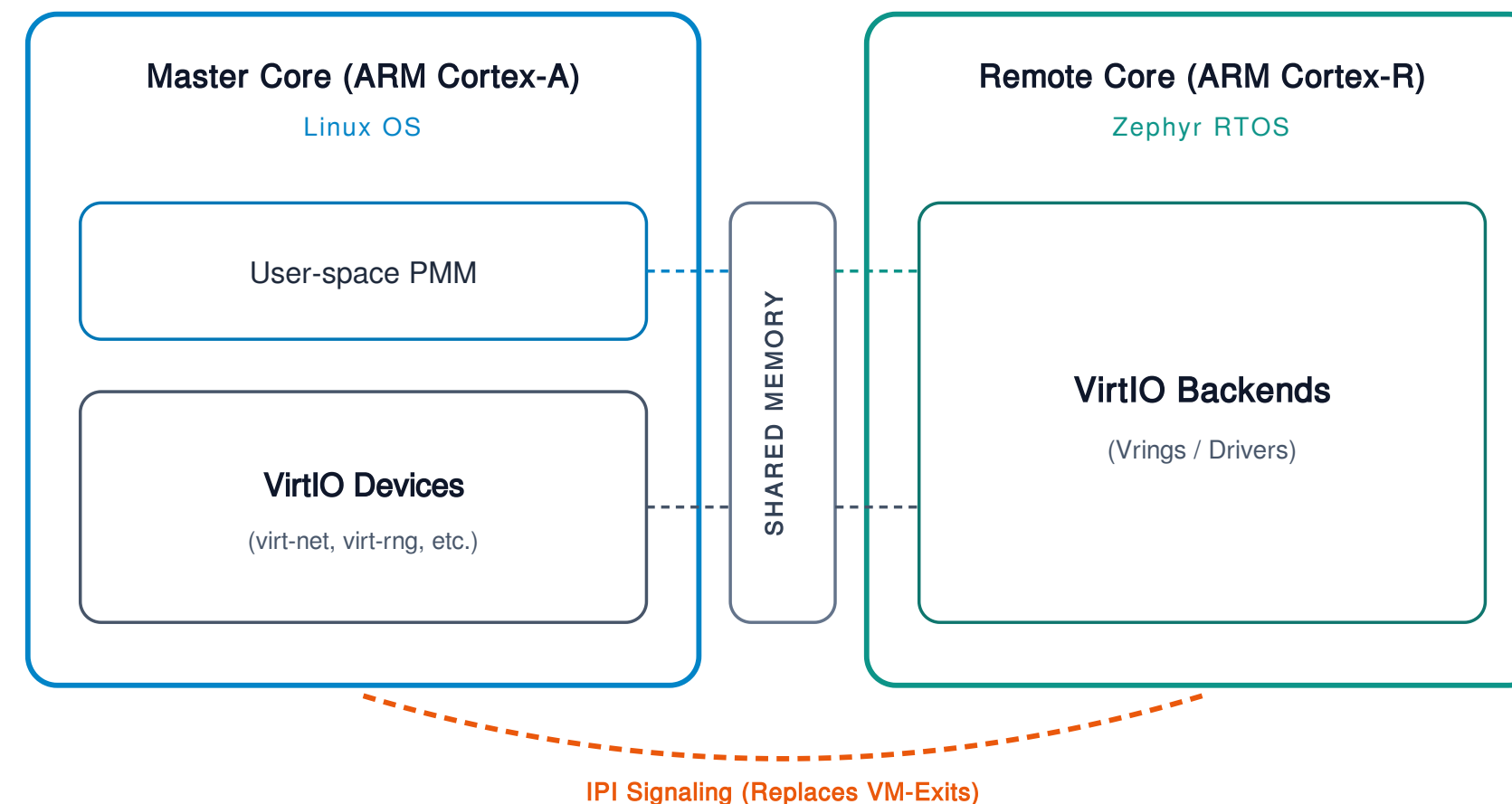
For IPC between heterogeneous processors in Asymmetric Multiprocessing (AMP) systems, the conventional approach is to use the **OpenAMP framework**, which implements the **RPMsg protocol over shared memory**.



Traditional OpenAMP Architecture

Beyond RPMsg: Hypervisorless VirtIO for AMP Architectures

Bypassing heavyweight hypervisor VM-exits, the **hypervisorless VirtIO** approach leverages a lightweight, user-space **Physical Machine Monitor (PMM)** on the Linux master core to expose standard **virtual devices** directly to **Zephyr RTOS**.

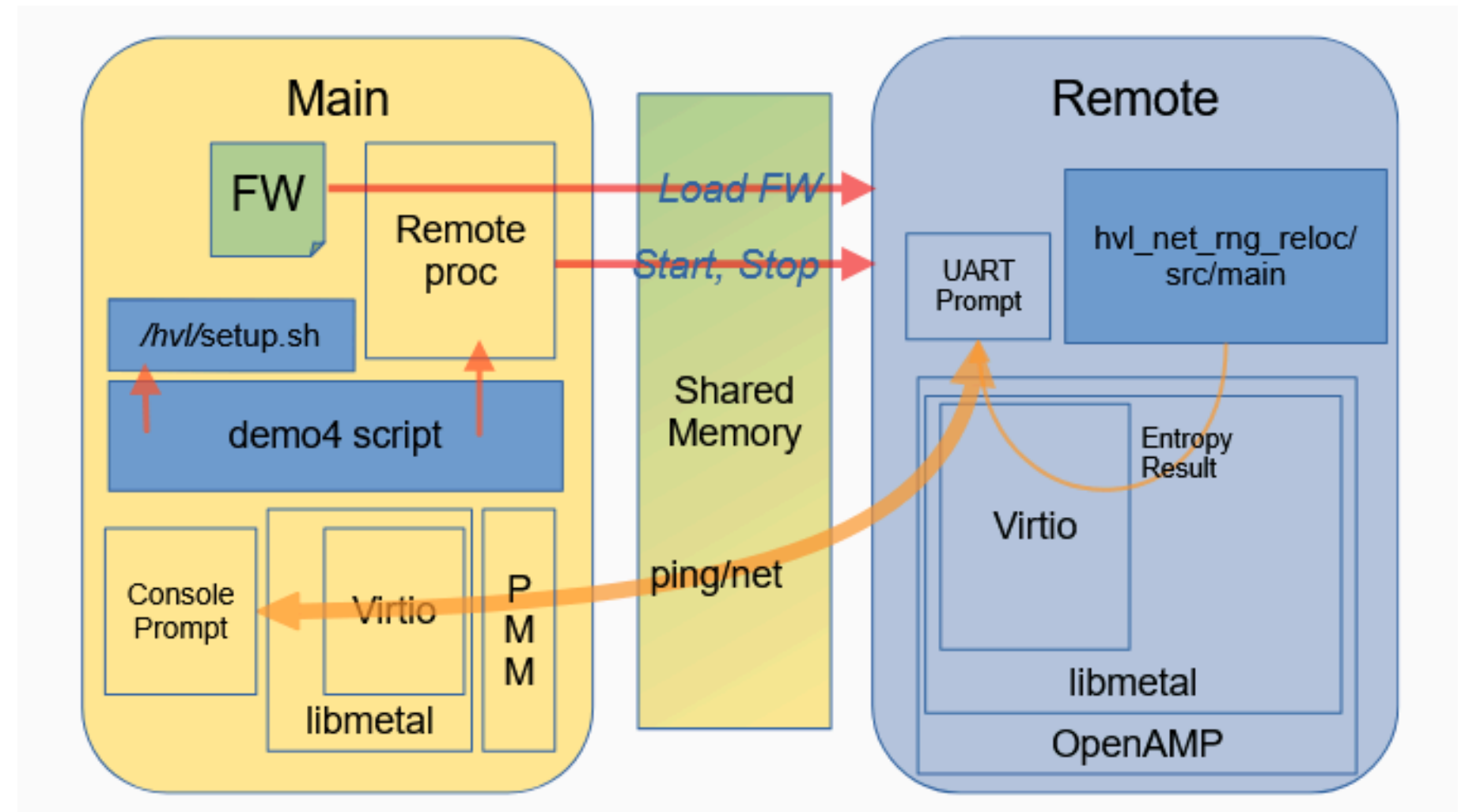


Hypervisorless VirtIO Architecture

Reference Platform & Hardware Emulation

To support reproducible experiments, the Xilinx Zynq MPSoC (**ZCU102**) is emulated with **QEMU** in a containerized **Docker** environment.

On this platform, the OpenAMP **demo4** framework runs an unmodified Zephyr guest equipped with standard **virtio-net** and **virtio-rng** devices, connected through a static physical shared-memory bridge.

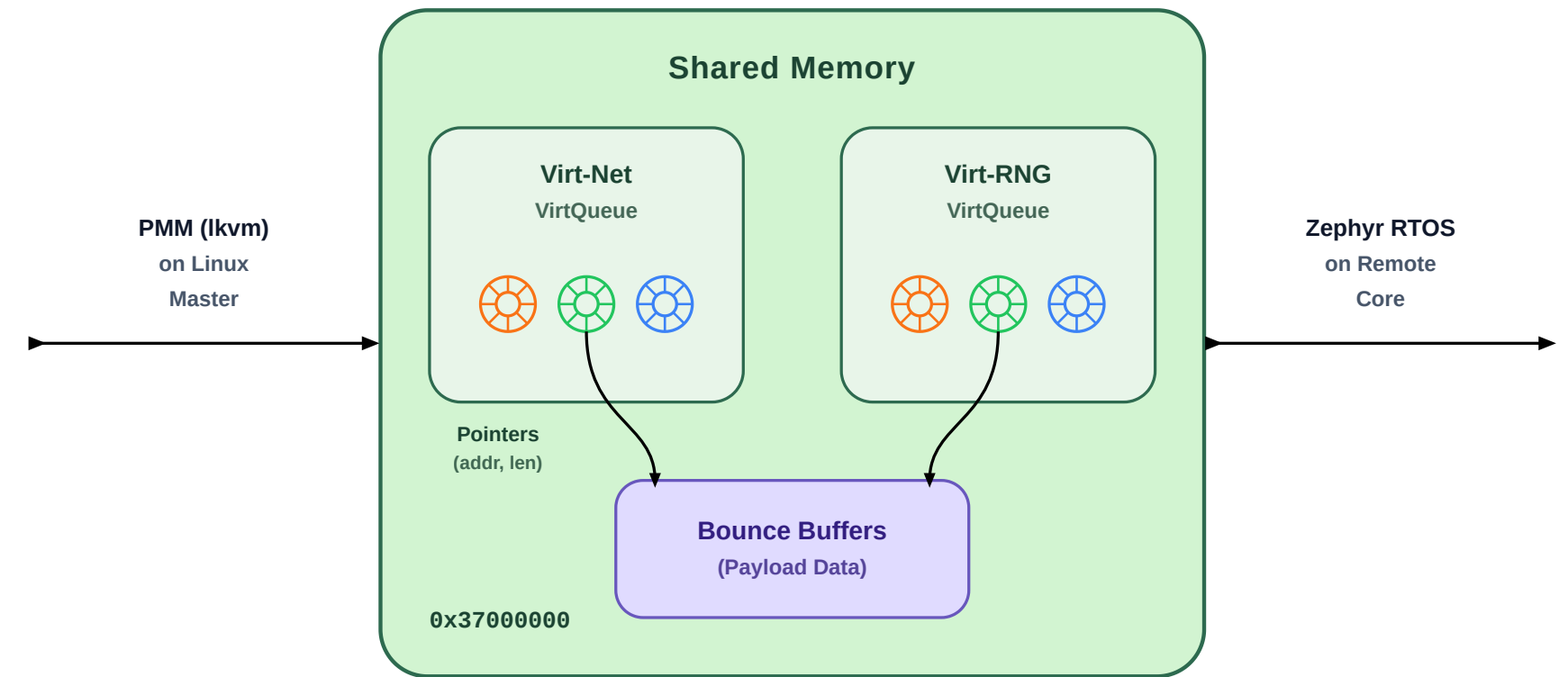


Emulated Environment Overview

Data Path Optimization: VirtQueues & Bounce Buffers

To enable hypervisorless communication, the design places **VirtIO-MMIO** in statically shared physical memory, granting both the Linux PMM and Zephyr RTOS symmetric access.

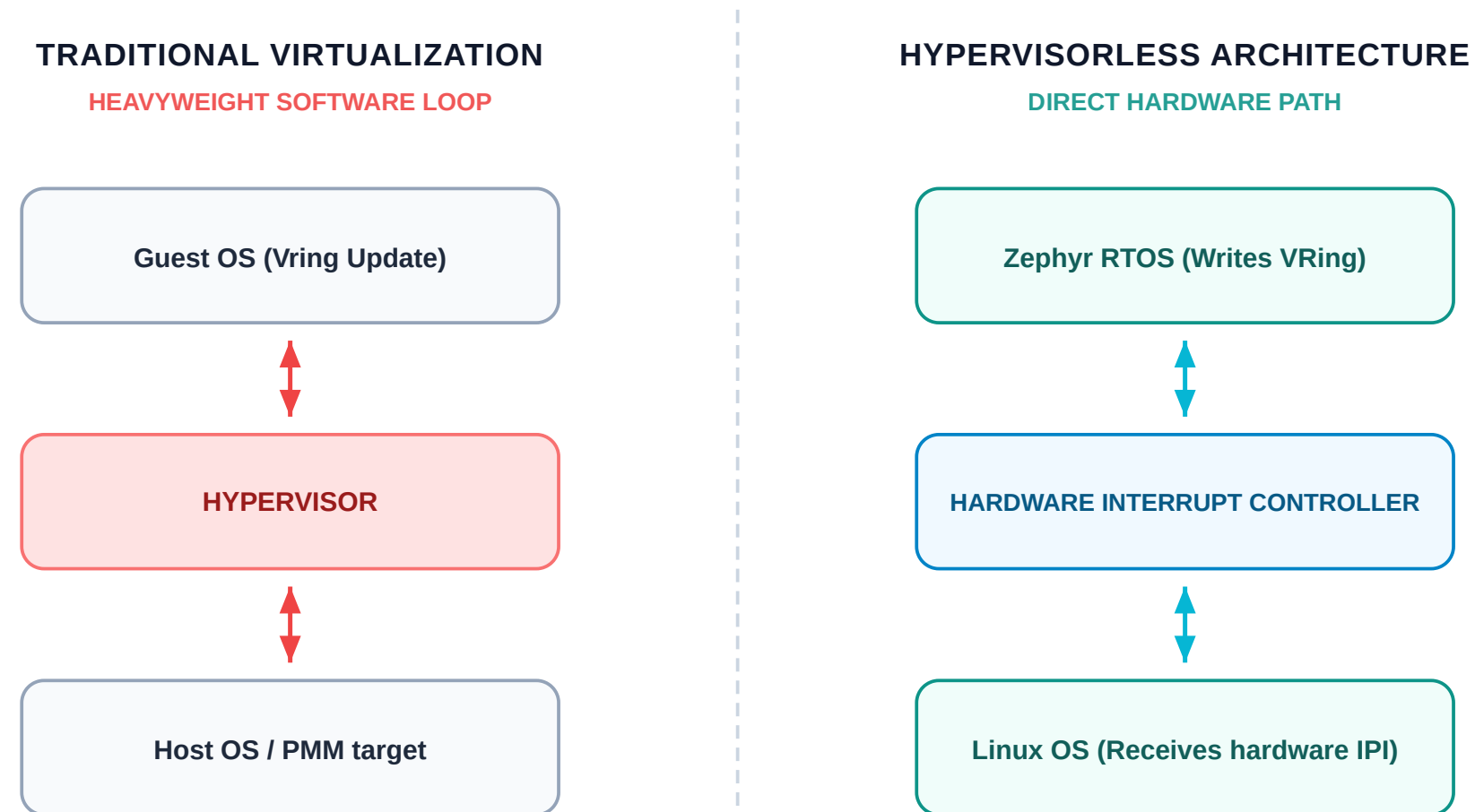
The control path still follows the conventional **three-VRing** layout, whereas the data path avoids software translation traps entirely by pointing Descriptor Ring entries directly to pre-allocated, **physically contiguous bounce buffers**.



Detailed View of Shared Memory

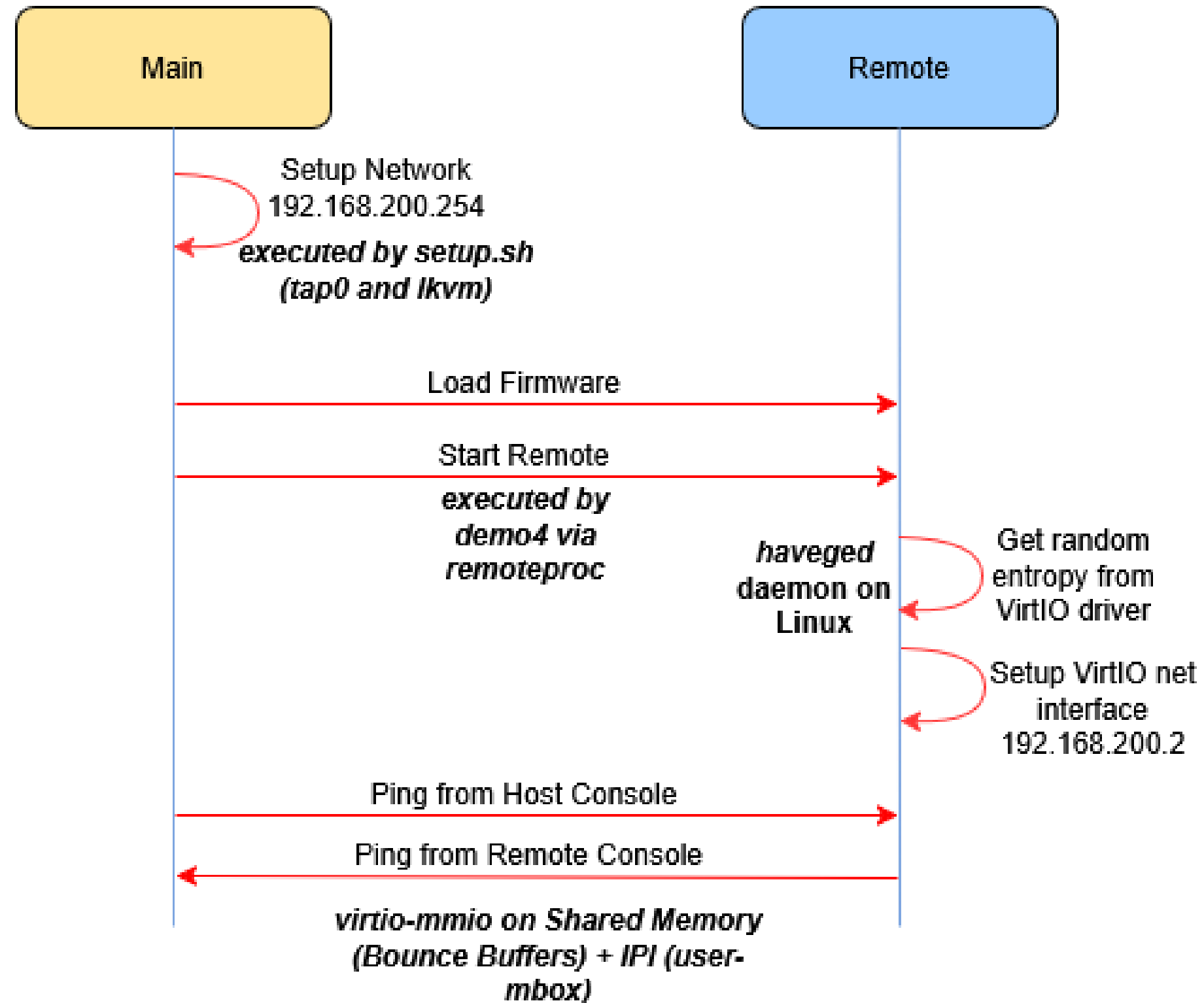
From VM-Exits to Hardware IPIs

Because both operating systems run natively on **bare-metal**, the **conventional** hypervisor-mediated **VirtIO doorbell mechanism**, which relies on VM exits, **cannot be used**. The architecture therefore replaces it with direct physical **Inter-Processor Interrupts (IPIs)** for signaling notifications, **eliminating software traps** entirely.



VM-Exits and IPIs Paradigms

Execution Flow



Mapping Theory to Code

```
50 static void xlnx_ipi_isr(const struct device *dev)
51 {
52     uint32_t isr, imr;
53     uint32_t i;
54
55     isr = READ32(dev, IPI_ISR);
56     imr = READ32(dev, IPI_IMR);
57
58     for (i = 0 ; i < 32; i++)
59     {
60         if (imr & (1 << i)) /* masked */
61             continue;
62         if (!(isr & (1 << i))) /* not active */
63             continue;
64         /* ACK interrupt */
65         WRITE32(dev, IPI_ISR, (1 << i));
66     }
67
68     #ifdef HVL_VIRTIO
69     virtio_mmio_hvl_cb_run();
70     #endif
```

Implementation of the IPI handler in xlnx_ipi.c

```
28 SECTIONS
29 {
30     shmem0 (NOLOAD): {
31         __shmem0_start = .;
32         KEEP(*(.shared.vring.*));
33         __shmem0_pool_start = .;
34         __shmem0_end = __shmem0_start
35     } > shmem0
```

Linker script section highlighting VirtQueue allocation

- **Static memory mapping:** Standard dynamic allocation is bypassed; VirtQueues are forcefully allocated at the shared physical address `0x37000000` via directives in the custom linker script, as well as bounce buffers.
- **Direct hardware signaling:** The absence of traditional software traps (VM-exits) is compensated by physical Inter-Processor Interrupts (IPIs), triggered via a direct memory write to the IPI_TRIG hardware register

Static Memory Inspection

```
(qemu) xp /32xw 0x37000000
0000000037000000: 0x74726976 0x00000001 0x00000004 0x4d564b4c
0000000037000010: 0x00000000 0x00000000 0x00000000 0x00000000
0000000037000020: 0x00000000 0x00000000 0x00001000 0x00000000
0000000037000030: 0x00000000 0x00000080 0x00000002 0x00001000
0000000037000040: 0xaabbaabb 0x00000000 0x00000000 0x00000000
0000000037000050: 0x00000000 0x00000000 0x00000000 0x00000000
0000000037000060: 0x00000000 0x00000000 0x00000000 0x00000000
0000000037000070: 0x00000004 0x00000000 0x00000000 0x00000000
(qemu)
```

Memory dump at shared memory base address

- **0x74726976**: "v i r t" string in ASCII
- **0x00000001**: VirtIO device version (Version 1).
- **0x00000004**: Device ID. According to the VirtIO specification, ID 4 corresponds exactly to the **Virt-RNG** (entropy) device.
- **0x4d564b4c**: In ASCII, this translates to "**L K V M**", the Vendor ID. This provides irrefutable proof that the device was created by the user-space lkvm PMM.
- **0xaabbaabb**: Value that matches the macro defined in Zephyr's driver, for a custom acknowledgement of the hypervisorless paradigm

Dynamic Payload Interception

ping -p **52** -s 128 -c 500 192.168.200.2

Hexadecimal value
corresponding to
letter “**R**” in ASCII

- **The cost of isolation:** The Zephyr driver uses its private SRAM, which is inaccessible to Linux; the transport therefore mandates an explicit copy (memcpy) of the packet into the intermediate Bounce Buffers.
- **Dynamic verification:** By suspending QEMU and scanning the Descriptor Ring, the physical pointer to the Bounce Buffer was located, visually proving the transit of the custom payload within the shared memory

```
(qemu) xp /128c 0x370078e8
00000000370078e8: '\x19' '\x03' '\x00' '\x00' '\x19' '\x03' '\x00' '\x0
0' '\x00' '\x00' '\x8a' '9' '\x05' '\xa7' 'o' '\x08'
00000000370078f8: '\x00' '\x00' '\x00' '\x00' '\x00' '\x00' '\x08' '\x0
0' 'E' '\x00' '\x00' '\x9c' '\x00' '\x00' '\x00' '\x00'
0000000037007908: '@' '\x01' 'h' '\x0f' '\xc0' '\xa8' '\xc8' '\x02' '\x
c0' '\xa8' '\xc8' '\xfe' '\x00' '\x00' 'O' '\x90'
0000000037007918: '\x02' 'U' '\x00' 'l' 'v' '\xb3' 'G' '\x0b' 'R' 'R' '
R' 'R' 'R' 'R' 'R'
0000000037007928: 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' '
R' 'R' 'R'
0000000037007938: 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' '
R' 'R' 'R'
0000000037007948: 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' '
R' 'R' 'R'
0000000037007958: 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' '
R' 'R' 'R'
0000000037007968: 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' '
R' 'R' 'R'
0000000037007978: 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' '
R' 'R' 'R'
0000000037007988: 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' 'R' '
R' 'R' 'R'
0000000037007998: 'R' 'R' 'R' 'R' '\x00' '\x00' '\x00' '\x00' '\x18' '\
x00' '\x00' '\x00' '\x96' '\xe1' '?' '\x00'
```

Custom payload found in bounce buffer

Custom Firmware Instrumentation

```
2 "2nd UART, R5_0"
0x37
0x23
0xb3

=== CUSTOM INSTRUMENTATION RNG ===
Requested and received 32 entropy bytes from Linux PMM!
Hexadecimal values read by VirtIO-RNG:
0a 1f 9d 37 bc fd 20 ac
4b 43 ac 5f ee 48 b3 db
68 1b 48 5d 84 5d 80 ce
cb 9b 2e e3 37 23 b3 00
=====

main:121 - get_entropy test passed
```

Entropy bytes print on Zephyr remote core

- **Custom cross-compilation:** Generated the zephyr.elf personalized firmware, forcing the compiler to exclusively link the experimental hypervisorless frontend modules to resolve dependency conflicts.
- **Frontend alteration:** The boot sequence of the virt-rng driver on Zephyr was modified to explicitly request and print a larger payload (32 bytes of entropy).

Latency Analysis in Zync UltraScale+ with QEMU



~2.4 ms difference in maximum latency, which proves the architectural trade-off: lacking RPMsg's true zero-copy, the processor is forced to spend extra clock cycles explicitly copying the larger payload into the shared bounce buffers (**explicit memcpy**).

Contextualizing Time and Emulation Limits

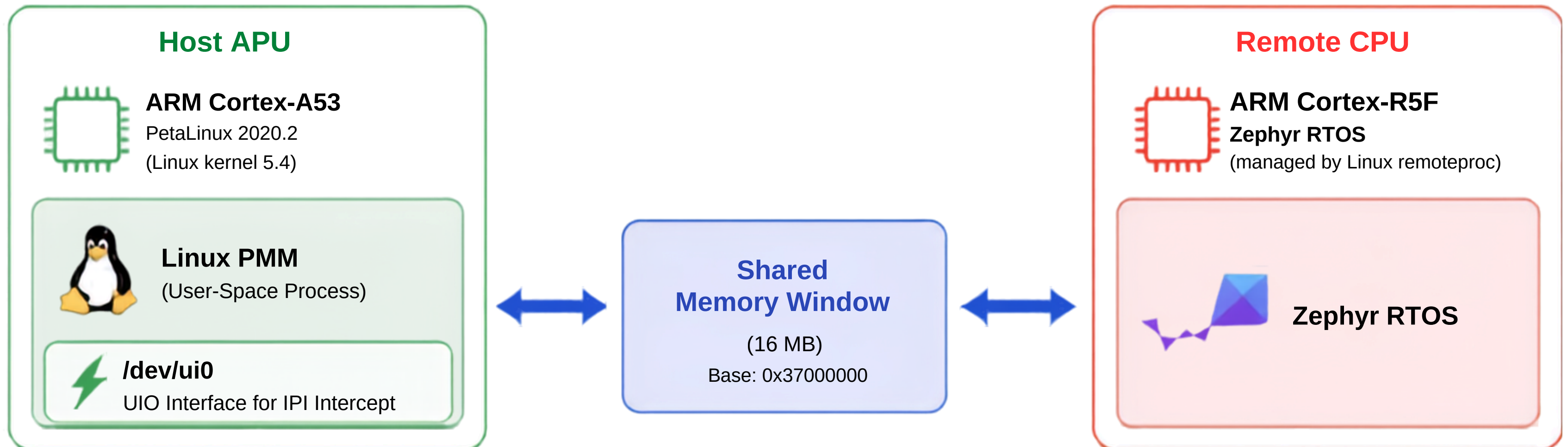
It is mandatory to state that the **timing measurements** obtained in this experiment are **not representative** of absolute real-time hardware performance.

As the execution environment relies entirely on QEMU, the recorded ping latencies are **heavily distorted** by the host OS scheduler, background processes, and the virtualization overhead of the Docker container.

Therefore, the results successfully demonstrate the **functional behavior** and the relative trends of the hypervisorless VirtIO architecture, but they **cannot** be used to assess adherence to industrial real-time deadlines.



Physical Deployment on Xilinx ZCU102



From Emulation to Physical Silicon

What Worked

TRNG node issue mitigated: the missing TRNG device tree node was worked around by enabling several flags, allowing the firmware to initialize successfully.

Zephyr firmware built successfully: the Zephyr application was cross-compiled and produced as ELF.

Firmware loaded via remoteproc: the zephyr.elf image was loaded into the remote Cortex-R5 using the Linux remoteproc framework without errors.

What Caused the Stall

Unable to access shared memory: lkvm terminates with a SIGBUS (BUS_ADRALN) fault.

Memory attribute conflict: the UIO driver maps the region as Device Memory, which is incompatible with ARM64 paired-store (STP) instructions.

Evaluated Workarounds

Mitigation Strategy	Target Layer	Outcome & Root Cause
User-Space Redirection (/dev/mem)	User space (LD_PRELOAD, svc #0)	Failed: Direct mapping defaulted to Device Memory attributes, causing a BUS_ADRALN fault and precluding hardware IPI signaling.
Forced O_SYNC Attribute Promotion	User space / Kernel MMU	Failed: Kernel access policies (STRICT_DEVMEM) prevent promotion to Write-Combining.
Dynamic Driver Rebind (uio_dmem)	Kernel / sysfs	Failed: The target devicetree lacks a dedicated CMA coherent memory reservation pool.

Therefore, end-to-end deployment on physical hardware remains an **open challenge**

Real-Time Relevance: Where It Wins, Where It Waits



Massive Software Reuse: By elevating VirtIO directly over shared memory, the Zephyr RTOS interacts with the Linux backend as if it were interacting with physical hardware. Standard VirtIO drivers can be used without writing custom, proprietary RPC protocols from scratch.

True Bare-Metal Determinism: Removing the heavyweight hypervisor eliminates VM-exits and hypervisor scheduling overhead. The **Cortex-R5** core runs natively on bare-metal, achieving desired determinism for its local tasks

Direct Hardware Signaling: Uses physical **Inter-Processor Interrupts (IPIs)** for direct core-to-core signaling.



Security Vulnerabilities: The PMM operates in Linux user-space with direct access to the pre-shared memory pool. Without strict hardware separation, a compromised "rich OS" could expose the RTOS memory to external vulnerabilities

Explicit Memory Copies: The lack of shared-memory **awareness** in standard frontend drivers mandates an explicit memcpy into the shared bounce buffers, **limiting throughput**.

Inherited Jitter on the Communication Path: While the RTOS is deterministic, the inter-processor data exchange is not. Waking up the Linux PMM relies on the Linux OS scheduler (kernel module) introducing latency and the typical jitter of general-purpose operating systems.

Questions & Answers

